

IBM Backend 進度 與下一階段實驗規劃

Hybrid Quantum-Classical DQN for CartPole-v1

目的：說明 IBM backend 已完成的串接、真實 QPU 結果、與 Classical/Quantum Torch 實驗的對齊方式，並提出可成功完成正式比較的步驟。

報告日期：2026-08-13

Branch：quantum-part--seanliu

1. 執行摘要

46	2/2	93	34
VQC 可訓練參數	硬體動作一致	ISA 電路深度	雙量子位閘

目前已打通完整鏈路：已訓練 Torch VQC checkpoint → 同一組 normalization 與參數 → Qiskit exact 驗證 → IBM ISA transpilation → Runtime Estimator → results JSON。真實硬體對兩個挑選狀態皆做出相同 action，但 State 0 的硬體 margin 僅 0.025，因此現在應稱為 preliminary hardware evaluation，而非完整 robustness 證明。

最重要的定位

IBM QPU 目前不是 DQN 的訓練 backend。訓練仍由 PyTorch 可微分 statevector 完成；IBM backend 負責訓練完成後的固定狀態推論與噪聲穩健度評估。這與黑客松時間和 QPU queue 限制相容，也能維持公平比較。

下一個里程碑

將兩狀態流程泛化為 12 個固定狀態（high/medium/low exact margin 各 4 個，action 0/1 平衡），先本機驗證，再用單一 Runtime job 批次執行，產生 action agreement、Q-value error 與 margin 分層結果。

2. 三條實驗路徑如何對齊

項目	Classical Zoo	Quantum Torch	IBM QPU evaluation
用途	主要 classical baseline	訓練 compact VQC policy	測試已訓練 VQC 的硬體噪聲
Q-network	256×256 MLP	4-qubit、3-layer data-reuploading VQC	同一個 VQC checkpoint 與電路
可訓練參數	67,586	46	不重新訓練；固定 46 個參數
訓練 backend	PyTorch CPU	Torch differentiable statevector	不適用（post-training inference）
共享流程	CartPole、normalize_obs、DQN loop、replay、target network、greedy eval	同左	沿用相同 normalization、selected states 與 action 定義
主要輸出	reward/solved/時間/參數	reward/solved/時間/參數	expectations/Q-values/action agreement/error

公平比較的核心

Classical 與 Quantum Torch 的訓練比較只替換 Q-function approximator；環境、觀測正規化、訓練迴圈和評估規則保持一致。IBM 評估則固定已訓練 policy，只替換 expectation-value execution backend，因此回答的是 hardware robustness，而不是訓練速度或 sample efficiency。

3. IBM backend 已完成進度

階段	狀態	產出 / 證據
帳戶與 backend 連線	完成	可取得 operational QPU；使用 ibm_marrakesh (156 qubits)
ISA-compatible VQC	完成	原始 depth 24 → ISA depth 93；two-qubit gates 34
Runtime smoke test	完成	Job d9ujrck98n5s7392v2q0；驗證提交與取回結果
Trained-policy local preview	完成	Torch 與 Qiskit exact max $ \Delta Q = 9.375e-05$
Submission manifest	完成	2 states × 2 observables；44 circuit values/state
首次 policy job	保留為部分結果	d9uks0l35hes73fk2fv0；發現 element-wise broadcasting
Broadcast 修正	完成	observable (1,2) × parameter (2,1,44) → result (2,2)
完整 hardware job	完成	d9ulamt35hes73fk3400；action agreement 2/2
可重現摘要	完成	JSON + Markdown summary 與分析程式已提交 branch

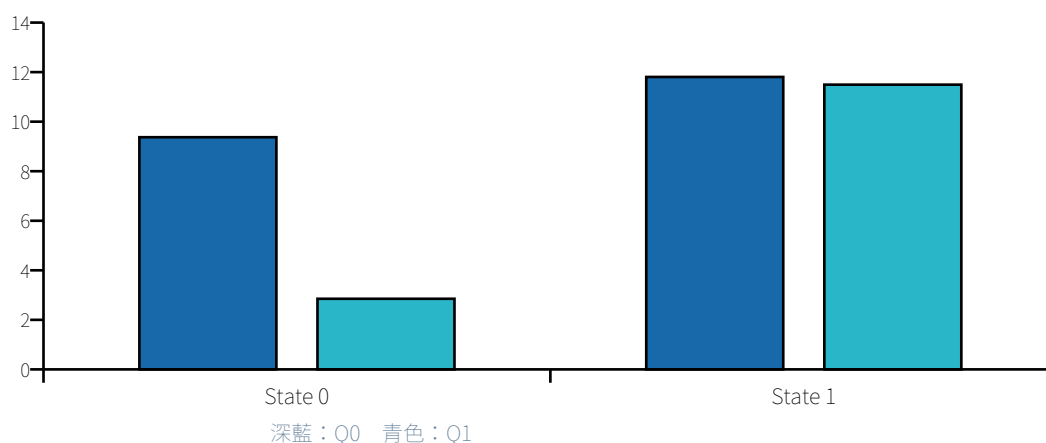
Data re-uploading 的位置

Data re-uploading 已包含在 VQC 建構中：4 維 observation 經 input_scale 後，在 3 個 variational layers 重複編碼，共形成 12 個 input angles；另外有 32 個 θ weights 與 2 個 output scales，總計 46 個可訓練參數。

4. 真實 IBM QPU 結果

State	Qiskit exact Q	IBM hardware Q	Exact action	Hardware action	Hardware margin
0	(101.719, 89.470)	(92.346, 92.321)	0	0	0.025
1	(87.361, 101.617)	(75.556, 90.123)	1	1	14.567

IBM hardware 相對 exact simulation 的 |Q-value error|



結果解讀

兩個 action 都一致 (2/2)，mean absolute Q-value error 為 8.881，最大誤差 11.805。State 1 的 action margin 仍大；State 0 的兩個 hardware Q-values 幾乎相等，表示其 action 對 finite-shot/hardware noise 很敏感。

限制：樣本只有兩個，而且是從單一 greedy episode 中挑選的 high exact-margin states；目前不能推論整條 policy 在狀態空間上的 hardware robustness，也不能把 100% 當成具統計代表性的準確率。

5. 與 Classical / Quantum 實驗的現況比較

實驗	參數	Greedy eval mean	Solved	Elapsed	可回答的問題
Classical Zoo	67,586	500.0	是	83.0 s	強 baseline 與解題能力
Quantum Torch	46	201.44	否	349.0 s	極低參數下可學到多少 policy
IBM QPU (2 states)	固定 46	不適用	不適用	含 queue	固定 policy 在硬體上的 action 保真度

目前可支持的結論

Compact data-reuploading VQC 僅用 46 個 trainable parameters，遠少於 67,586 參數的 Classical Zoo MLP；它在 Torch statevector 上學到非平凡 policy (greedy mean 201.44)，但尚未解出 CartPole。真實 IBM QPU 對兩個固定狀態保留了 action，顯示硬體推論鏈路可行；尚未證明整體 robustness。

注意：`comparison.json` 中的 classical run (greedy 123.51) 不是最強 Zoo baseline。正式報告應以 `classical\_zoo\_run.json` 的 greedy 500.0 作主要 classical 比較，並清楚標示不同設定。

6. 下一階段：成功跑出可比較的正式實驗

步驟	本機 / QPU	具體工作	完成條件
1. 建立狀態集	本機	從相同 greedy evaluation seeds 收集候選 states；依 exact margin 分 high/medium/low	12 states；每層 4 個；action 0/1 平衡
2. 精確驗證	本機	Torch 與 Qiskit StatevectorEstimator 比較 Q-values	所有 states action 一致；差異 $\leq 1e-4$
3. 送出前預覽	本機 + backend read	固定 checkpoint hash、參數順序、ISA metrics、broadcast shape	不送 job；manifest 可審查
4. 批次硬體執行	IBM QPU	單一 PUB 批次執行 12×2 expectations，precision 0.1	保存 job ID；得到 (12,2)
5. 穩健度分析	本機	overall/margin-group agreement、MAE、max error、action flips	JSON、CSV/圖表、Markdown 摘要
6. 重複執行	IBM QPU	額度允許時，以相同 states 與參數再跑一次	hardware-to-hardware action consistency

為什麼先做 12 states

比兩個 state 更能觀察 decision boundary，又可控制 QPU 成本。high-margin 檢查明確決策是否保留；medium-margin 反映一般狀態；low-margin 專門測試 noise 是否造成 action flip。成功後再擴展到 20–50 states，而不是直接把完整 CartPole episode 放在 QPU queue 中逐步互動。後者延遲高、成本大，也不適合三天黑客松的可重現實驗。

7. 執行與成果交付清單

類型	應保留檔案	用途
訓練模型	results/quantum_policy.pt	固定 46-parameter VQC checkpoint
Torch 訓練結果	results/quantum_run.json	reward、時間、參數、greedy evaluation
IBM 前置驗證	artifacts/ibm/validation/ibm_policy_preview.json	selected states 與 exact validation
IBM 送出 manifest	artifacts/ibm/validation/ibm_policy_submission_preview.json	checkpoint hash、ISA 與參數 binding
IBM 正式結果	artifacts/ibm/core/ibm_policy_evaluation.json	expectations、Q-values、actions、errors
報告摘要	artifacts/ibm/core/ibm_policy_hardware_summary.{json,md}	GitHub 與簡報可直接引用

重要操作原則

每次真正送出前先產生 preview；確認 `qpu\_job\_submitted=false`、checkpoint SHA-256、state 數量、observable 數量與 broadcast result shape。送出後立即保存 job ID；終端中斷時使用 `--resume JOB\_ID`，不要重新執行提交指令，以免產生重複 job。

建議接續 commit

下一個 commit 應只包含 12-state 泛化程式、local preview 與分析模板；真正 QPU 結果完成後再以獨立 commit 加入，方便追蹤實驗版本。

結論：IBM backend 技術串接已完成。接下來的重點不再是『能否連線』，而是建立公平、批次化、可重現且具分層分析的 hardware robustness experiment。